The City Brain inside the triplet

Three layers, one loop. The **Twin** is the simulated city, the **Guardian** says which readings can be believed, and the **City Brain** decides and acts. Figure 1 is the Brain's part of the global diagram: every box is a component that exists and runs, and the arrows are the ways they hand work to each other.

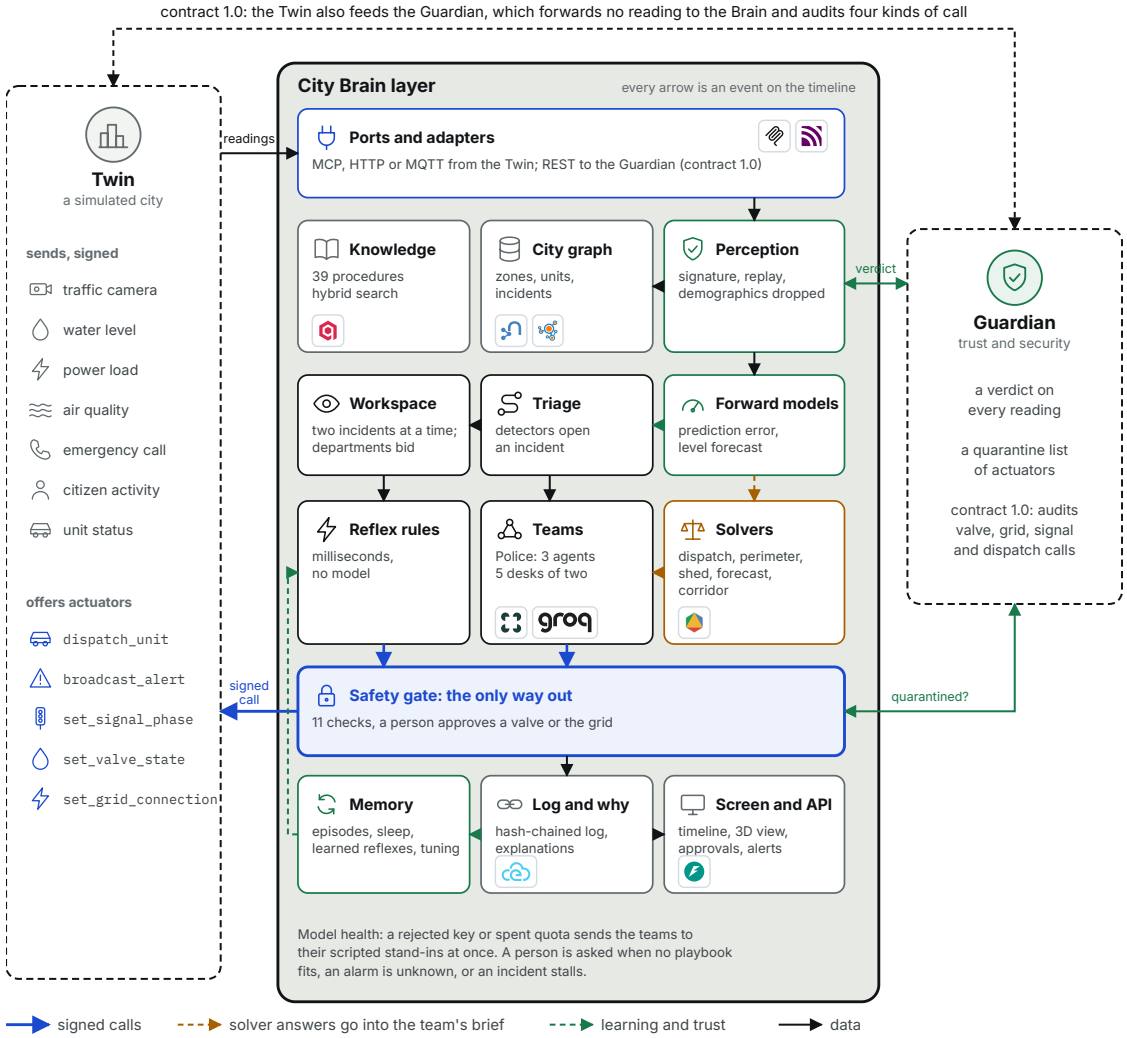


Figure 1. Global architecture. Readings enter at the top and actions leave only through the gate. The dotted line across the top is the Guardian team's design, where the Twin also feeds the Guardian. Dashed amber arrows show where a solver's answer enters a team's brief; dashed green arrows show the learning loop.

Link	How it talks	What keeps it safe
Twin to Brain	MCP tool `list_readings` (also HTTP and MQTT); city layout from `get_topology`	contract version, HMAC signature, sequence number, ranges
Brain to Guardian	REST, 2 s deadline. Our contract: `POST /verdicts` with the reading, `GET /quarantine/{id}`. The Guardian team's 1.0: `POST /v1/verdicts/request` by sensor and sequence number, `GET /v1/containment`	a silent Guardian means no verdict, so no action on readings it has not judged
Twin to Guardian	the Guardian team's contract only: the Twin pushes `POST /v1/readings` (or the Guardian pulls). The Brain is not on this path	HMAC on every reading; the Guardian checks sequence and timestamp
Brain to Twin	signed actuator calls through the Twin's MCP tools; with the Guardian team's Guardian, valve, grid, signal and dispatch calls go through its `POST /v1/actuator-calls`, which audits them before forwarding; results come back as readings and `unit_status`	the gate: 11 checks, trust reference, idempotency, human approval
Any partner to the screen	CloudEvents over SSE; a partner may push `twin.*` or `guardian.*` events	`brain.*` pushes are refused; scoped tokens off loopback

One scenario, end to end: crime and pursuit

A robbery is caught on camera in Downtown (scenario `t14`). The trace below is the run committed in `logs/t14-20261108T153858Z.jsonl`, at normal speed; times are measured from the robbery report. Read it left to right: the same event is touched by every layer and by most components of the Brain, and each hand-off is an event on the timeline.

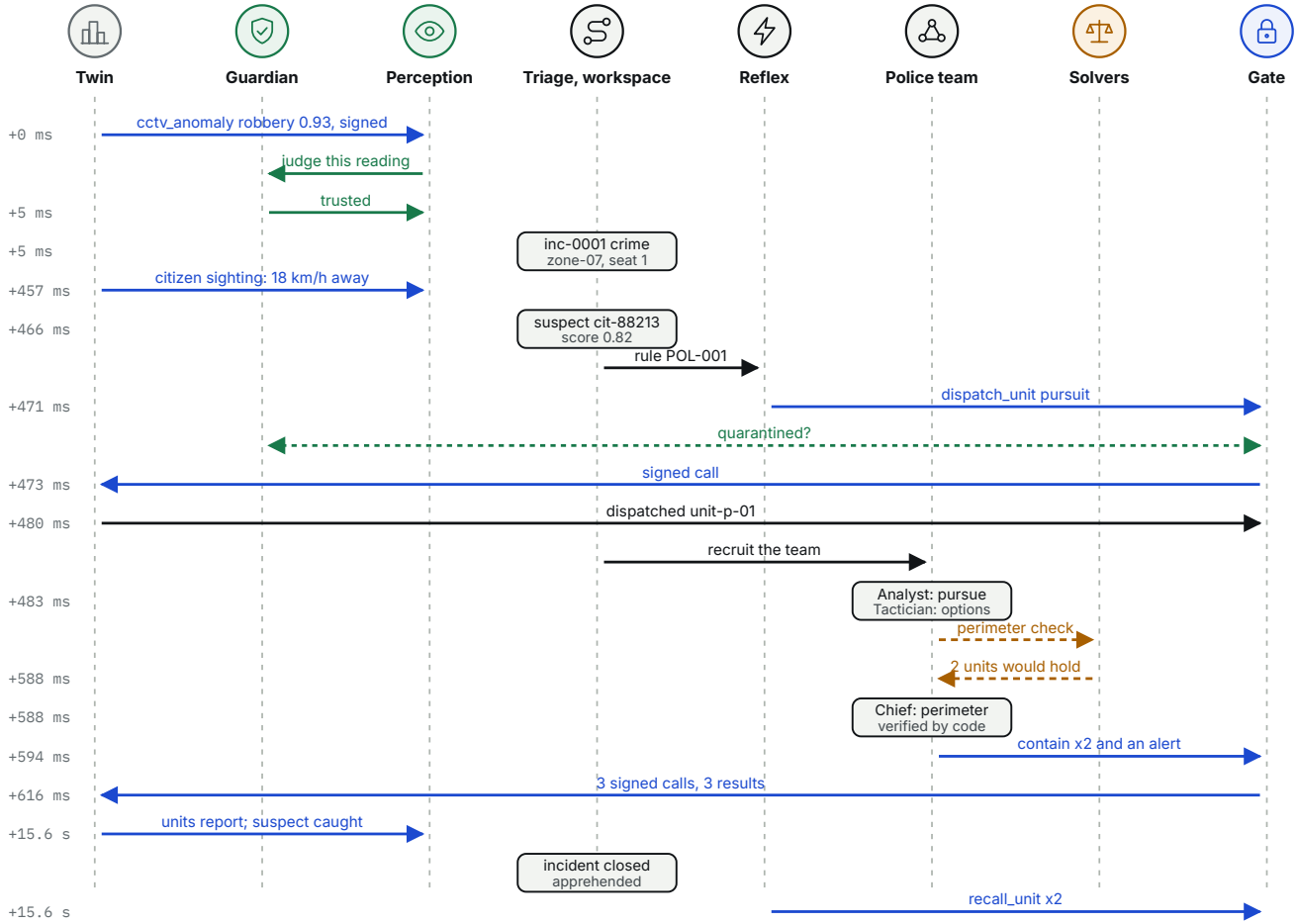


Figure 2. The reflex dispatches 15 ms after the reading that completes the evidence (10 ms after its verdict); the team's fuller plan follows about 120 ms later and passes the same gate. The perimeter solver checks the team's plan (2 units hold, 2 are used). Recorded against our stand-in Guardian, to which the Brain sends the reading itself. With the Guardian team's contract 1.0 the Brain asks by sensor and sequence number and the dispatches pass through the Guardian's audit.

What this shows

Trust first. The camera reading is acted on only after the Guardian's verdict; the second reading (a citizen running away at 18 km/h) lifts the suspect's score to 0.82.

Two speeds. A rule dispatches the first unit at once; the Analyst, Tactician and Chief then place a perimeter, and code verifies it before anything is proposed.

What it would look like if it went wrong

A fake flood (t13): the Guardian marks the reading untrusted and the gate refuses the warning and the valve call. **A layer down (t15):** the Guardian stops answering; the Brain says so once and acts on no reading the Guardian has not judged (every call still needs a trusted verdict).

How the layers were made to fit

The Brain is built so that a partner can be replaced without touching the Brain: everything that depends on the Twin's naming, units or speed is configuration, not code, and every behaviour is agreed with the partners in the Interface Card.

One contract. The Data Dictionary messages, one example of each in `contracts/examples/`, JSON Schemas exported by one command, a four-edit rule for any change.

Trust is shared. The Guardian judges; the Brain scales trust by the verdict's confidence and never learns from an untrusted reading.

One way out. Only the gate signs a call; agents get read-only tools; a valve or the grid needs a person.

Every layer runs alone. The mocks of the Twin and the Guardian ship with the Brain; `brain conformance` checks a real partner against the same contract.

Partners differ. `brain onboard` proposes renames and units for a Twin that names things differently and never edits config.

Failure is a state. A silent Guardian, a missing Twin, a spent model quota and an unknown alarm each have a named event and a defined behaviour.

Evidence, not claims. 1,114 tests, 5 Test Book scenarios, 60 held-out variants, a hash-chained log per run, and `brain rehearse` as the freeze check.

One command. `make full` builds one image and starts the Brain, both stand-ins, Neo4j and Qdrant.

The messages that cross the layers

Message	From, to	Examples in the repository
Sensor reading envelope	Twin to Brain	`reading.water_level.json`, `reading.cctv_anomaly.json`, and five more
Trust verdict	Guardian to Brain	`verdict.json` (status, confidence, expiry)
Actuator call and result	Brain to Twin and back	`actuator_call.*.json` (six actions), `actuator_result.*.json`
Approval	person to Brain	`approval.json` (signed, bound to the call, single use)
Timeline event	Brain to screen and partners	`timeline_event.json`; 55 event types in `docs/asyncapi.yaml`

When something fails, the layers stay joined

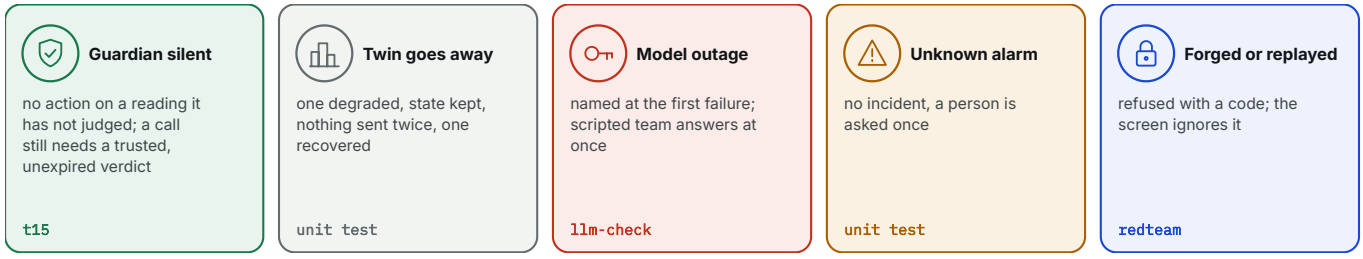


Figure 2b. Each failure has a name, a defined behaviour and a scenario or command that shows it.

Where the Brain still depends on the other teams

The Brain has run end to end only against stand-ins for the Twin and the Guardian. Which sensors move together and whether same-type sensors in one zone are independent are still to be measured on the real Twin (`docs/TWIN-HANDOVER.md`); each is a switch that is built and waits for the answer.

Built to connect to a team it has never met

In Phase 2 the triplets are reshuffled. A new Twin or Guardian means a new adapter or a changed configuration file, never a change to the decision logic.

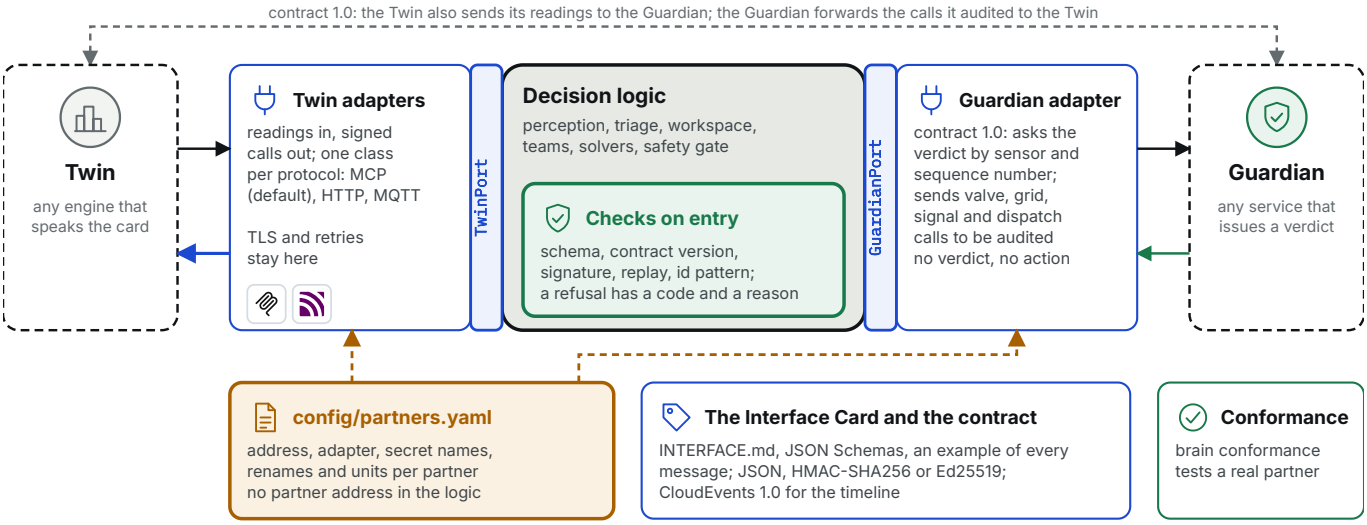


Figure 2c. Everything outside the two ports can be replaced; the logic cannot tell which partner it talks to.

The five things the jury looks for, and where to check each

Requirement	What the repository does	How to check it
Logic separate from partner I/O	The logic reaches partners only through `TwinPort` and `GuardianPort` (`ports.py`). Protocols live in `adapters/`, one class each: MCP, HTTP and MQTT for the Twin, REST for the Guardian.	`tests/test_layering.py` fails if logic imports an adapter or the reverse; `tests/test_twin_mqtt.py` runs `t14` over MQTT.
A complete Interface Card	`INTERFACE.md`: what the Brain receives and sends, an example of each message, how to repoint it, how it authenticates. Long form: `docs/INTEGRATION-brain.md`.	`tests/test_contracts.py` validates every example in `contracts/examples/` and verifies each signature.
Settings in configuration	`config/partners.yaml` holds each partner's adapter, address, secret variable name, renames and units. Secrets come from the environment; code defaults are loopback addresses.	A test requires the YAML to equal the defaults; `brain onboard` proposes edits for a renamed Twin.
Standard protocols and formats	MCP, REST and MQTT; JSON Schema; HMAC-SHA256 or Ed25519 over canonical JSON; Server-Sent Events and CloudEvents 1.0 described by AsyncAPI; TLS; OpenTelemetry.	`contracts/schemas/`; `docs/asyncapi.yaml` (55 event types).
Validation and clear errors	Every reading is checked on entry; a refusal carries a code and a reason (below) and goes to the timeline. A bad or missing verdict means no action; a failing Twin tool is named in the error.	`brain conformance` checks a real partner; `brain redteam`: 24 attacks, all refused.

What a partner is told when it sends something unexpected

The partner sends	Code	The Brain's answer, as the code returns it
unknown contract version	`bad_contract_version`	contract_version '2025-01-01' is not supported; this Brain accepts ['2026-09-17']
a field missing (zone_id)	`schema_invalid`	reading envelope is invalid: zone_id: Field required
no signature	`signature_missing`	wl-01-01#1: provenance.hmac is required
a value changed in transit	`signature_invalid`	wl-01-01#1: provenance.hmac does not match the payload
the same reading twice	`replay_or_out_of_order`	wl-01-01: sequence_no 1 is not greater than the last seen (1)
a number sent as text	`value_invalid`	wl-01-01#1: value for type 'water_level' is invalid: level_m: Input should be a valid number, unable to parse string as a number

Table 2. Run through the Brain's own perception code when this report is built. Not shown yet: the Brain has run only against mocks of the Twin and the Guardian; the HTTP and MQTT adapters are agreed but not yet run against a partner.

2. The City Brain

Written by the City Brain team · repository folder `triplet-03/brain/` · contract `2026-09-17`, Guardian contract 1.0 · key question: "What should we do?"

2.1 Role and goals

The Brain reads the Twin's live sensor readings, detects incidents (crimes, fires, accidents, jams, floods, grid overloads), asks the Guardian whether each reading can be trusted, chooses a response and carries it out through the Twin's actuators. It acts only on readings the Guardian has marked trusted, and only through the safety gate, the one component allowed to call an actuator.

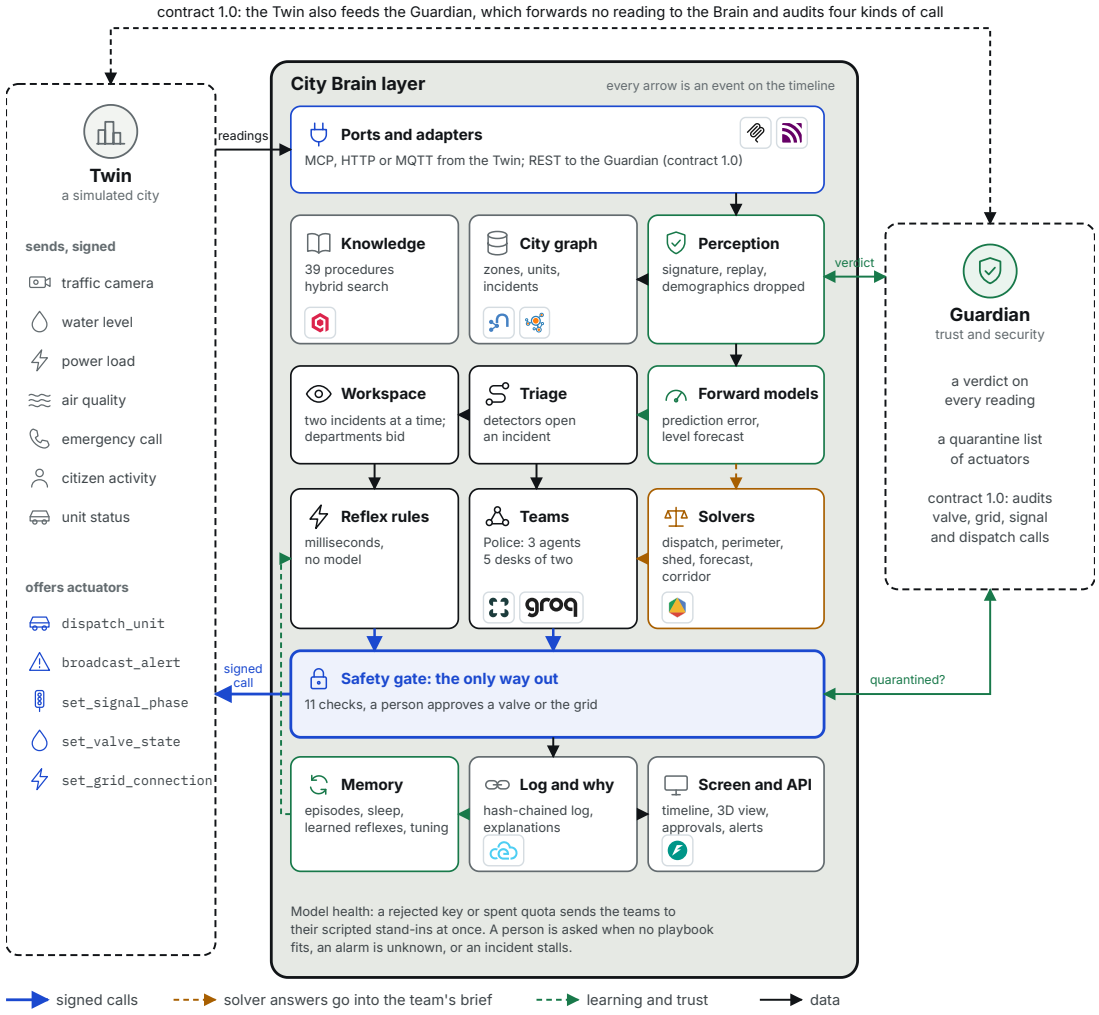


Figure 2.1. The Brain's components. Readings enter through the adapters at the top and actions leave only through the gate. With the Guardian team's contract 1.0 (dashed line) the Twin also feeds the Guardian, and valve, grid, signal and dispatch calls reach the Twin through the Guardian's audit. Amber dashes: a solver's answer enters a team's brief; green dashes: learning.

Goals, and how each is shown

Goal	How the Brain meets it	Shown by
Act only on trusted data	every call carries a `trust_ref` to a trusted, unexpired verdict	T13: a fake flood, nothing sent (Listing 2.1)
One way out	rules, agents and solvers propose; only the gate sends	`brain redteam`: 24 of 24 attacks refused
Fast first, then thorough	a reflex acts at once; the team's plan follows, checked by code	T14: reflex in 15 ms, team plan 123 ms later
Safe when a part fails	no verdict, no action; a scripted team replaces a lost model	T15: Guardian silent, nothing more sent
Every step explained	each decision, hold or refusal is a named event with its reason	chained log per run; the "why" API

All five Brain scenarios (T12 to T16) pass, with their logs in `logs/`; 1,114 tests; 60 of 60 held-out variants decide as their base scenario.

2.2 Architecture

Figure 2.2 follows one reading through the components of Figure 2.1, in eighteen steps from the Twin to an action and its explanation. Each card names the component, the technology that runs it and, in blue, the brain function that inspired it. The analogy guides the design; the Brain does not model a brain.

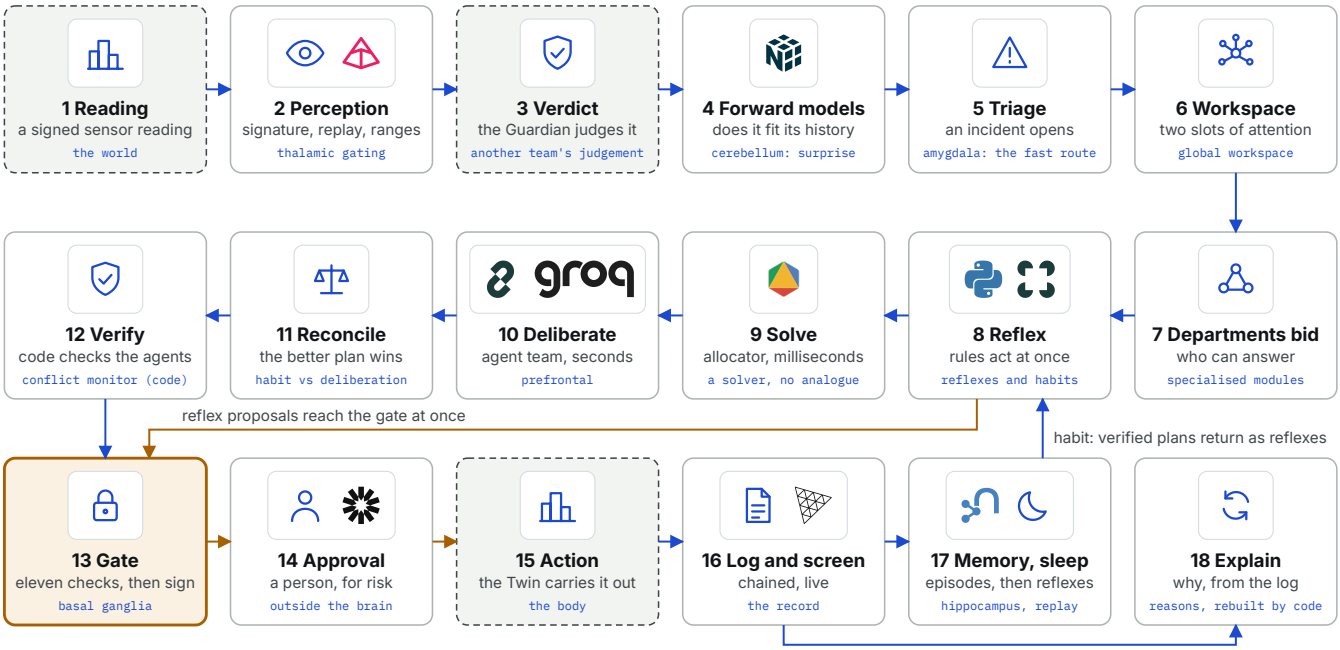


Figure 2.2. From a reading to a decision. Amber arrows lead to the gate: reflex proposals reach it at once, and the team's plan follows through the same gate. A plan the team makes the same way twice returns as a reflex after the sleep phase.

2.2.1 Incident detection

Perception checks the contract version, the HMAC signature, the sequence number and the value ranges, and refuses a bad reading with a code. **The Guardian's verdict**, requested with a 2 s deadline, decides whether the reading may count. **Forward models** learn each sensor's normal range from trusted readings only. A reading far outside it is flagged as a surprise, and the reflex it would trigger waits for a second source; a Kalman tracker forecasts when a river will reach the flood line. **Triage** runs the departments' detectors to open or join an incident, and the **workspace** gives two incidents at a time the team's attention, ranked by severity, trust, surprise and urgency.

When an alarm matches no department, an incident has no procedure, or an incident stalls, the Brain emits `brain.human.needed` and asks a person.

2.2.2 Decision method

Rules (13, in six packs) answer in milliseconds with no model. **Agent teams** weigh the case: for Police an Analyst, a Tactician and a Chief on LangGraph, using Groq models with `--llm` and a scripted stand-in otherwise or when the model fails; the other departments use a two-role desk. **Solvers** compute what should not be guessed: OR-Tools min-cost flow for dispatch, CP-SAT for a perimeter and for load shedding, Dijkstra for a green corridor and the Kalman forecast for floods. They run before the agents and their answers go into the agents' brief, so a model that never calls a tool still reads them.

Code then verifies the agents' plan (`verify_case`, `verify_plan`, `verify_desk`) before it reaches the gate.

Departments

Department	Incidents	Acts through	Reasoning
Police	crime	`dispatch_unit` (pursuit, contain), `broadcast_alert`, `recall_unit`	Analyst, Tactician, Chief
Fire, Ambulance	fire, accident	`dispatch_unit`, `broadcast_alert`	desk: Analyst, Chief
Traffic control	jam	`set_signal_phase` (hold green, also for an ambulance's route)	desk
Disaster response	flood	`broadcast_alert`, `set_valve_state` (a person approves)	desk
Utilities	overload, smog	`broadcast_alert`, `set_grid_connection` (a person approves)	desk

A department is three files (detectors and actuators, reflex rules, a playbook): a new kind of incident is a new pack.

2.2.3 Choosing an actuator

The Brain picks an actuator in three steps. The incident type decides which department may act and with which actuator; a solver, or the nearest free station, picks the device; and the gate decides whether the call goes out. No other code path calls the Twin.

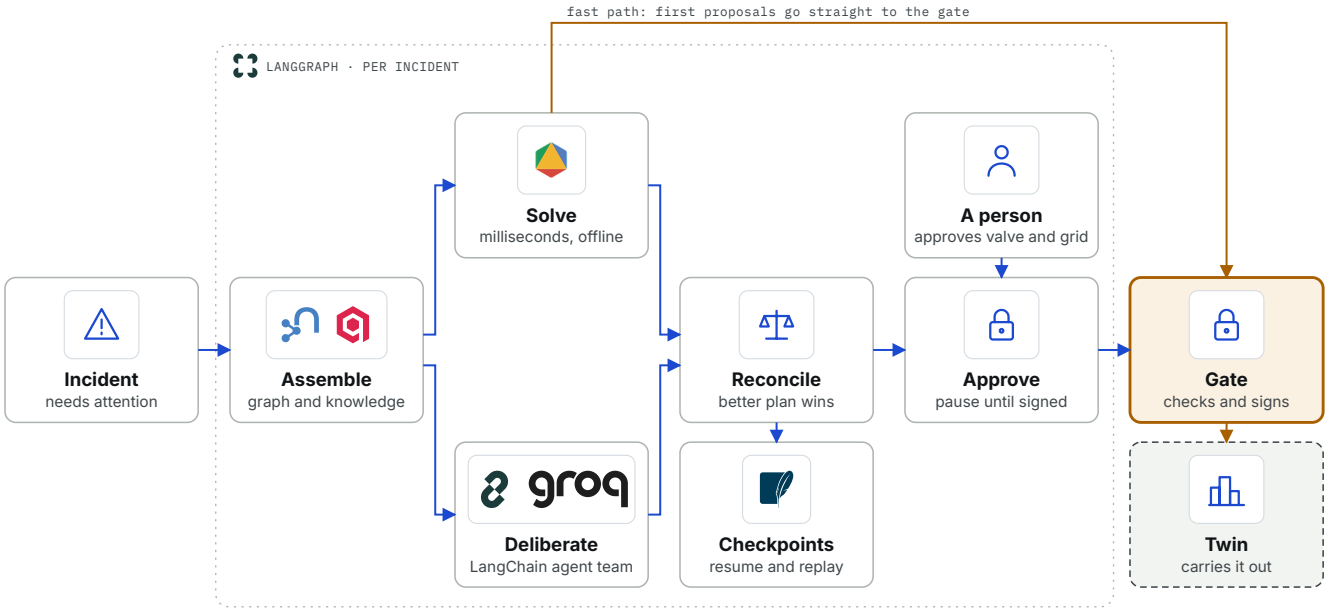


Figure 2.3. The planning graph for one incident (LangGraph). The solver and the agent team work in parallel, the better plan is kept, a valve or grid call pauses for a person, and each step is checkpointed to SQLite so a run can resume and be replayed.

Station or device

The nearest free station by hop distance, never a quarantined or reserved one. In T16 the Guardian quarantines a station, the gate refuses calls to it with reason `quarantine`, and another station is sent. When two incidents need the same kind of unit, the dispatch solver plans both together, and its choice is used only if it costs less: for two collisions it may send the far ambulance to the first so that the near one stays free for the second. On 100 random cities its plans cost 17% less than nearest-first (20.39 against 24.63) and match brute force (`brain optimise-bench`).

Grid cut or signal

A cascade analysis weighs what each grid cut would black out, with a hospital weighted 10, and the shed solver picks the cheapest set of cuts that removes the excess load. A corridor holds the signals on an ambulance's route at green. A dispatched unit's `unit_status` readings close the incident, and an arrest recalls every unit still out.

Table 2.1. The safety gate's eleven checks, in the order they run on every proposal. The first that fails stops the call and its reason goes to the timeline; quoted reasons are from the canonical logs. A call that passes is signed (HMAC, key id, one-time nonce) and sent; an approved valve or grid call runs every check again.

#	Check	Refuses the call when	Reason logged
1	**schema**	the call does not match the contract	call does not match the contract: *field error*
2	**capability**	the Twin does not list the action	the Twin does not expose *action*
3	**allowlist**	the department may not use the action	department *name* may not call *action*
4	**incident open**	the incident is closed (recalls still pass)	incident *id* is closed (*outcome*)
5	**trust_ref**	no trusted, unexpired verdict covers its evidence	"no Guardian verdict referenced"
6	**evidence**	too few trusted sources for its impact level	"high-impact action needs 2 independent trusted sources, found 1 (wl-01-01)"
7	**human approval**	a valve or grid call has no signed approval yet	a high-impact action needs a person's approval...
8	**idempotency**	the same call was already issued	already issued for *key*
9	**cooling gap**	the actuator was driven too recently	*actuator* was last driven *n* s ago; the minimum gap...
10	**quarantine**	the device is quarantined or cannot be checked	"the Guardian has quarantined police-dispatch-05"
11	**budget**	6 actions per incident, or 20 per minute	incident *id* reached its limit of 6 actions

2.3 Tech stack and why

Everything runs offline on one laptop: the embedding model is in the Docker image, the screen loads nothing from the internet, and the scripted team answers when no model is reachable.

Table 2.2. The stack by job. Each logo belongs to its owner.

	Job and technology	Why
	Language and contracts Python 3.12, uv, Pydantic	One language for rules, agents and solvers; locked builds; every message a typed model that validates itself.
	Agents LangGraph, LangChain, Groq	Graphs that pause, resume and checkpoint, as the agents and human approval need; a scripted team answers when no model is reachable.
	Solvers OR-Tools, NumPy, NetworkX	Proven solvers with a certificate for answers that must be exact; the Kalman forecast; the city graph in memory.
	Storage Neo4j, Qdrant, SQLite	"Why was this unit sent?" in one graph query; hybrid search over 39 procedures; approvals and checkpoints. No decision waits on them.
	Interfaces MCP, FastAPI, MQTT, CloudEvents	MCP gives the Twin's tools schemas and discovery; REST is one question, one answer per verdict; SSE is one-way, replayable, library-free.
	Operations Docker, OpenTelemetry, three.js, pytest	One command starts the whole city; spans follow a reading through every step; the screen works offline; 1,114 tests.

2.4 Interfaces in/out

Direction	Message	Transport and contract
In, from the Twin	signed sensor readings; the city layout	MCP `list_readings(cursor)` every 200 ms and `get_topology()`; plain HTTP or MQTT as alternatives
In, from the Guardian	a signed verdict per reading; containment	contract 1.0: `POST /v1/verdicts/request` by sensor and sequence number, HMAC checked; `GET /v1/containment`
Out, to the Twin	signed calls for the six actions (`dispatch_unit`, `set_valve_state`…)	valve, grid, signal, dispatch via the Guardian's audited `/v1/actuator-calls`; broadcast and recall to the Twin directly
Out, to the screen and partners	timeline events; the reasons for a decision; read-only tools	SSE `GET /events` (CloudEvents 1.0, 55 types in AsyncAPI); `/api/decisions/{id}/explanation`; MCP at `/mcp`

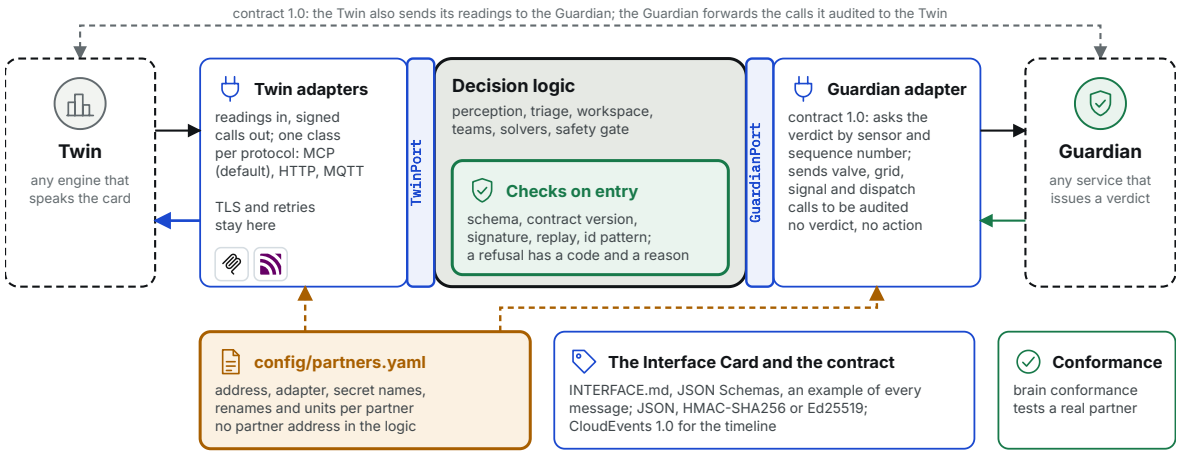


Figure 2.4. Where the logic ends and partners begin. Phase 2 reshuffles the triplets, so a new Twin or Guardian must be a new adapter class or an edit of `config/partners.yaml`, never a change to the decision logic, which reaches partners only through the two ports.

The jury looks for	Where the Brain does it, and the check that keeps it true
Logic separate from partner I/O	two ports in `ports.py`, one adapter class per protocol; `tests/test_layering.py` enforces it
A complete Interface Card	`INTERFACE.md`, an example of every message; `tests/test_contracts.py` validates each
Settings in configuration	`config/partners.yaml`, secrets from the environment; `brain onboard` maps a renamed Twin
Runs alone with mocks (R1)	`mocks/twin`, `mocks/guardian`; `brain conformance` runs 19 checks on a real partner

2.5 Security measures

The Brain treats every input as untrusted until it passes its checks, and every model answer as a proposal that code must verify. Valve and grid calls also need a person's signed approval. Threats are mapped to MITRE ATT&CK for ICS and, for the AI components, to MITRE ATLAS (`docs/SECURITY-MITRE.md`).

Listing 2.1. T13, a fake flood, from `logs/t13-20261108T153847Z.jsonl` (abridged). The gauge reads 1.80 m, the Guardian says untrusted, and the gate refuses both the warning and the valve.

```
15:38:51.522  brain.reading.received  wl-01-02  level 1.80 m
15:38:51.526  brain.verdict.received  verdict-000004  untrusted
15:38:51.526  brain.incident.opened   flood, zone-01, evidence untrusted
15:38:51.547  brain.action.blocked    broadcast_alert  trust_ref: no Guardian verdict referenced
15:38:51.548  brain.action.blocked    set_valve_state valve-01-01  trust_ref: no Guardian verdict referenced
15:38:51.975  brain.incident.closed   cleared: wl-01-01 still reads 0.30 m
```

Boundary	Control
Twin to Brain: readings	HMAC-SHA256 or Ed25519 over canonical JSON, key ids with expiry, a rising sequence number per sensor, schema and ranges; only allowlisted fields are kept, so demographic fields never reach a decision
Guardian to Brain: verdicts	schema check and the verdict's HMAC (contract 1.0); more independent trusted sources as impact rises
People and partners to the API	scoped bearer tokens off loopback (read, push, approve, admin), body and rate limits; a partner may push only `twin.*` or `guardian.*` events, and a forged `brain.*` event is refused
Brain to the language model	ids limited to safe characters before any prompt; read-only tools, with no actuating tool; models choose zone ids, not free text; code verifies the plan, then the gate checks it
Brain to Twin: calls	eleven gate checks; each call signed with its time and a one-time nonce; a valve or grid call needs a signed approval bound to its exact parameters and used once
Brain to its own disk	hash-chained log with a signed head; learning from trusted readings only; self-tuning limited to a listed set of parameters, each with a range

MITRE technique	Attack run against the code	Result
ICS T1692.002, forged reading	wrong key, no signature, a field edited after signing, a replayed sequence number, `alg: none`	refused every time
ICS T1692.001, T0849, forged call	an approval reused with changed parameters; the Twin pushing as the Guardian	refused
Enterprise T1190, T1078	no token, a forged token, a read token doing admin work, 28 path spellings with 7 methods	none got through
ATLAS AML.T0051, T0053	30 prompt-injection strings in sensor ids and values; a tool that tries to actuate	dropped before any prompt; no such tool exists

`brain redteam` runs 24 attacks; all were refused when re-run for this report. When the Guardian stops answering, the Brain acts on nothing it has not judged (T15), which is the Guardian's own fail-safe rule.

Reproduce: `uv run brain scenario t13 --inproc --speed 0`, `uv run brain redteam`, `uv run brain verify-log --scenarios`.

2.6 Known limitations

- **No joint run with the real Twin yet.** Against the live Guardian we ran the fake and real flood and T16. Next: run T12 to T16 against the Twin team's Twin.
- **The mock Guardian is scripted.** T13 and T16 show what the Brain does with a given verdict, not whether a real Guardian would issue it. Next: rerun both with `docker-compose.guardian.yml`.
- **Tuned on stand-ins.** Which sensors move together, and whether sensors in one zone are independent, waits for the real Twin's data. Next: record a normal day from it (`docs/TWIN-HANDOVER.md`).
- **TLS and the freshness limit are off by default.** Signatures still protect readings and calls. Next: set `guardian.tls` and `twin.max_age_s` on the live stack.
- **Guardian alerts are not read.** An actuator the Guardian flags as unreliable appears in its containment list, which the gate obeys; a `compromised_twin` alert does not reach the Brain. Next: read `WS /v1/ws/verdicts` in the adapter.

4. Test Book (the Brain's scenarios)

Five scenarios, each one trigger (`scenarios/triggers/tN.sh` or `uv run brain scenario tN`) and one hash-chained log in `logs/`, with the expected outputs in the scenario file. Together they cover the brief's four interaction rows; the Guardian team's T1 to T11 are in the shared `TESTBOOK.md`. The generated list, in the book's fields, is `brain/docs/TESTBOOK-brain.md`.

ID	Scenario	Interaction	Expected output
T12	A calm day	Twin → Brain	no incident, no actuator call: the Brain does not overreact
T13	A fake flood	Twin → Guardian, Brain → Guardian	the gauge is untrusted; the gate refuses the warning and the valve call; nothing reaches the city
T14	A crime and a pursuit	Twin → Brain, Brain → Guardian	reflex dispatch, a team perimeter checked and verified, units recalled at the arrest
T15	A flood, an attack and a layer down	all three	warning sent, valve held, forged report sends no police; once the Guardian is silent, one 'Guardian away' and no action on unjudged readings
T16	A quarantined police station	Brain → Guardian	the gate refuses calls to the quarantined station with reason `quarantine`; a different station is used